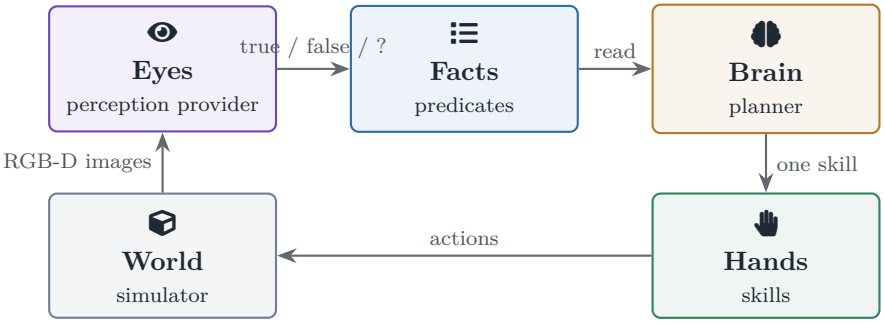
How the Robot Sees

Predicates, providers, planners — and what the `perception/grounding-evaluation` branch changes



Key idea

The robot never reads the simulator's answer key. It has to *see* the mug in camera images, turn what it sees into a few true/false/unknown **facts** (predicates), and let a **planner** choose the next **skill** from those facts. This branch measures how good those facts are and makes them better — without ever letting a wrong "true" through.

Contents

1 The big picture: eyes, facts, brain, hands **3**

2 Predicates: the facts the robot believes **4**
2.1 Three answers, not two . . . 4
2.2 The perception predicates this branch measures . . . 4
2.3 Where predicates are used, from start to end of a run . . . 5

3 Providers: who answers the questions **6**

4 Planners: the brain **7**
4.1 The full task as a chain of predicate gates . . . 7

5 How the camera provider actually sees **7**
5.1 From pixels to a fact . . . 7
5.2 Back-projection: one pixel to one 3D point . . . 8
5.3 Which camera answers . . . 8

6 What this branch builds: an evaluation loop **8**
6.1 Step 0: the scene viewer . . . 9
6.2 Step 1: labelled capture — public frames and a private answer key . . . 9
6.3 Step 2: the SAM 3 cache . . . 9
6.4 Step 3: the evaluator . . . 9

7 The fixes, one by one **10**
7.1 Fix 1: a perception-only camera . . . 10
7.2 Fix 2: a toppled mug is not “standing on the table” . . . 11
7.3 Fix 3: wait until the mug stops moving . . . 12
7.4 Fix 4: recognising the right-arm lift . . . 13
7.5 Fix 5: a mug on the floor is `off_table`, not toppled . . . 13
7.6 Probe: can a different SAM 3 question find sideways mugs? . . . 13
7.7 Speed: render only the frames that are used . . . 14
7.8 Smaller fixes along the way . . . 14

8 Why this matters: perception is the supervisor’s eyes **15**
8.1 What each fix gives the supervisor . . . 15
8.2 Idea 1: a supervisor must intervene safely . . . 16
8.3 Idea 2: a supervisor must notice failure . . . 16
8.4 Idea 3: no cheating — the supervisor must work without the answer key . . . 16
8.5 Telling a perception error from a supervisor error . . . 17
8.6 What the supervisor still cannot see well . . . 17

9 The commands, in order, and why **18**

10 Where we are, and what is next **20**

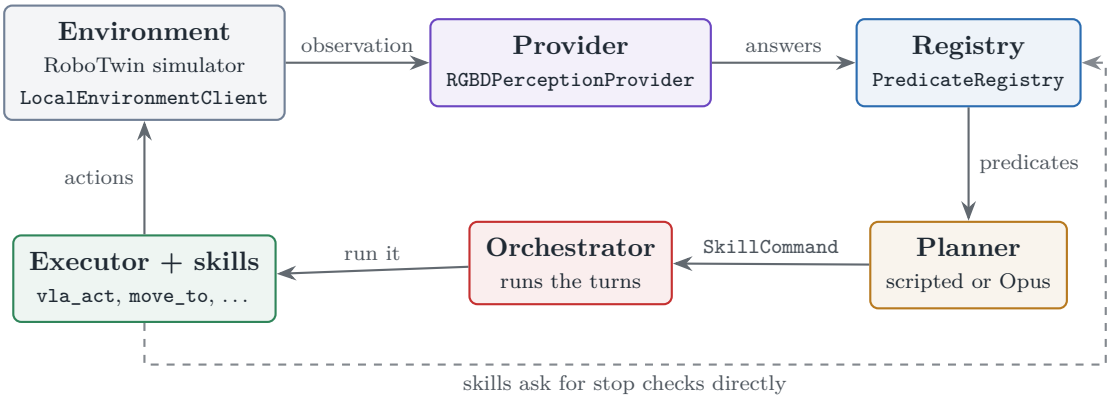
Glossary **21**

A Plan: recovering a toppled mug **22**
A.1 The recovery as a chain of skills and checks . . . 22
A.2 What exists and what is missing . . . 23
A.3 The perception pieces . . . 23
A.4 The motion piece: standing it up . . . 24

A.5 How to develop it safely 24
A.6 Effort and risk 24

1 The big picture: eyes, facts, brain, hands

A run of the hanging-mug task is a loop. Each *turn*, the robot looks, decides one thing to do, does it, and looks again.



In plain words

Think of a person tidying a kitchen with their eyes half closed. Their **eyes** (the provider) report short facts: "the mug is in my hand", "the mug is standing on the table". Their **brain** (the planner) only hears those facts and picks the next move. Their **hands** (skills) carry out the move, and stop early if a fact says "done" or "not safe". If the eyes say "I'm not sure", a careful person waits instead of guessing.

Piece	Its one job	File
Environment	Run the simulator, return RGB-D images	`harness/environment/local.py`
Provider	Turn images into facts	`harness/perception/grounding.py`
Registry	One place to ask for any fact	`harness/predicates/registry.py`
Planner	Choose the next skill	`harness/planner/`
Skills	Move the robot, with safety stops	`harness/skills/`
Orchestrator	Run turn after turn until done	`harness/runtime/orchestrator.py`

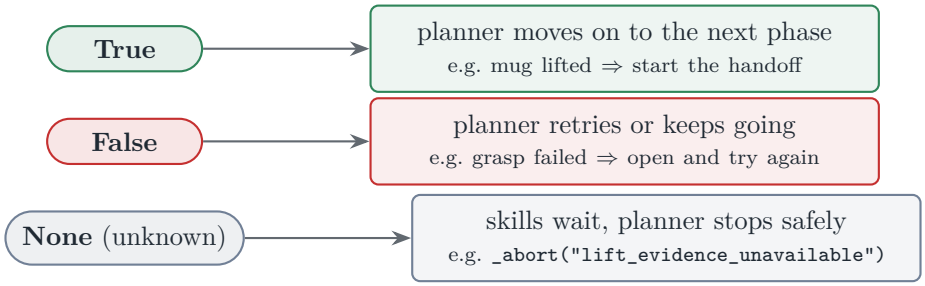
2 Predicates: the facts the robot believes

A **predicate** is a named yes/no question about the scene, such as "is the mug lifted?". Every answer is a `PredicateResult`:

```
PredicateResult(name="release_ready", value=True | False | None,
                source="perception", observation_sequence=432,
                reason="mug_silhouette_meets_local_table")
```

The `reason` is a short searchable string saying *why*. It is how we debug a wrong answer. The full list of names is `PREDICATE_NAMES` in `harness/predicates/registry.py`.

2.1 Three answers, not two



Careful

A wrong **true** is the dangerous mistake. A wrong `release_ready = true` makes the robot open its gripper while the mug is lying on its side — or in mid air. A wrong **unknown** only costs time. That is why every fix in this branch prefers "unknown" to a guess, and why we always report **precision** ("when it says true, is it true?") alongside recall.

2.2 The perception predicates this branch measures

Predicate	True when	Latches	Unlocks
`mug_lifted`	the mug has been lifted at some point	yes	HANDOFF
`release_ready`	the mug is upright and touching the table *now*	no	opening the gripper
`mug_staged_on_table`	`release_ready` after an earlier lift	no	the right-arm grasp
`right_mug_lifted`	the right gripper alone has held the mug up	yes	HANG

Latching means "once true, stays true". A milestone such as "the mug was lifted" remains a fact even if the mug is dropped later, and the planner keeps the phase it unlocked.

2.3 Where predicates are used, from start to end of a run

1. **Choose the source of facts**: camera provider, oracle, or none
   `run_skills.py: build_evidence()`
2. **Prime**: remember the mug's starting height ("lifted" is measured against it)
   `executor.py: predicates.prime()`
3. **Each turn**: observe, ask every predicate, give the answers to the planner
   `orchestrator.py: _predicates()`
4. **Planner reads them**: scripted `context.value("release_ready")`, Opus gets them in its prompt
   `scripted.py, anthropic.py`
5. **Skills stop on them**: `vla_act` stops at `mug_lifted`; the gripper only opens if `release_ready`
   `vla_act.py, analytic.py`
6. **End**: `task_success`, budget used up, or out of turns
   `orchestrator.py`

Key idea

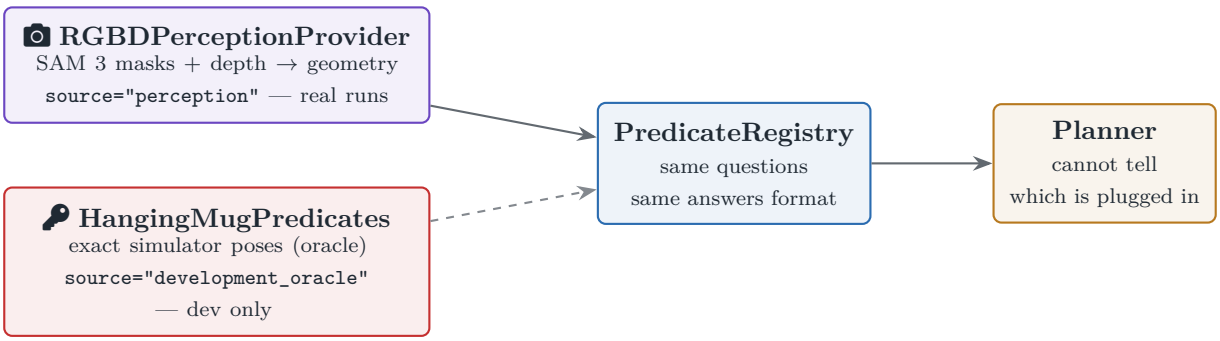
Camera predicates are checked at **turn boundaries** and when a skill checks its stop condition, not after every single action: SAM 3 is too slow for that. Answers are cached per observation, so the planner and a skill's stop check can never see two different answers for the same frame.

3 Providers: who answers the questions

The registry does not work anything out itself. It forwards every question to a **provider**:

```
provider = self.perception if self.perception is not None else self.hanging_mug
return provider.evaluate(name, observation, arm=arm)
```

Anything with a `source` and an `evaluate(name, observation, arm)` method is a provider. That makes providers **swappable**, like plugs in one socket:



Run with	What you learn
Oracle provider	Does the planner and skill logic work when facts are perfect?
Camera provider	Does it still work with real, imperfect perception?
No provider	Every answer is `None`; skills wait or refuse to act.

Careful

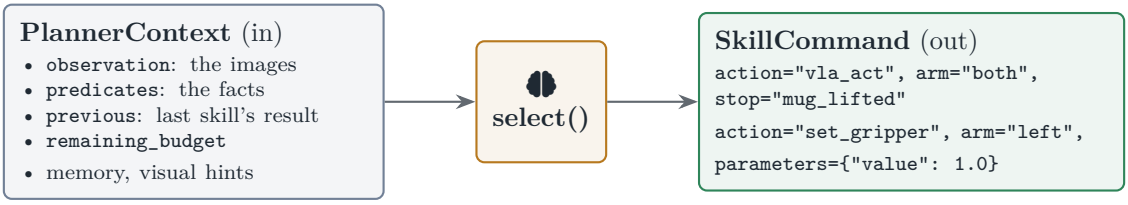
Name trap: inside `HangingMugPredicates`, `self.provider` is something else — the small function that fetches `MugEvidence` (exact positions and contacts) from the simulator. Same word, one level down. The registry refuses the oracle unless the run explicitly allows it, so it can never leak into an evaluation.

There is also a narrow third source, `RackSeatReleaseGate`, which answers only `rack_seat_aligned` and is asked first for that one question.

4 Planners: the brain

A planner has exactly one method:

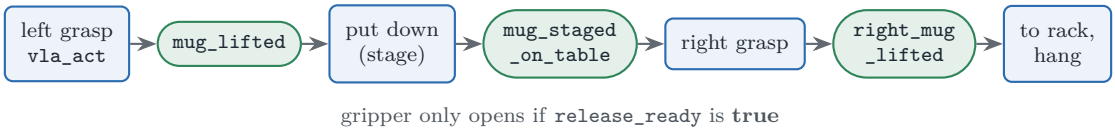
```
def select(self, context: PlannerContext) -> PlannerSelection: ...
```



The `stop` field must name a real predicate; this is how a decision by the brain becomes a safety stop inside the hands.

Planner	How it decides	File
`GraspRetryPlanner`	fixed script: approach, grasp until lifted, retry if the grasp failed	`scripted.py`
`HandoffPlanner`	scripted full task: left grasp, stage, right grasp, rack, hang	`handoff.py`
`AnthropicPlanner`	Opus sees images + predicates, replies with one checked skill	`anthropic.py`

4.1 The full task as a chain of predicate gates

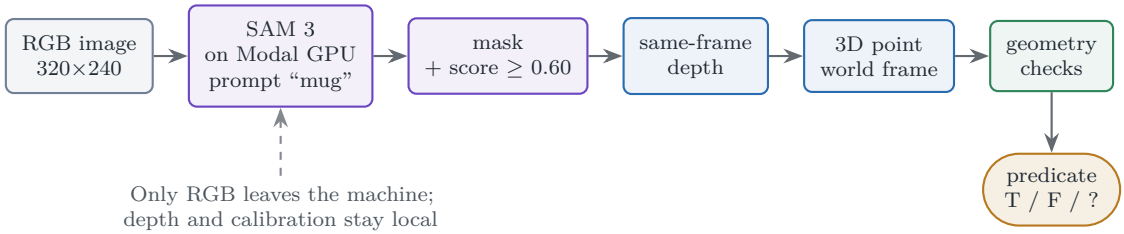


gripper only opens if `release_ready` is **true**

Each green gate is a predicate the camera provider must answer. If a gate is wrongly true, the robot skips ahead (for example, lets go of a toppled mug). If it is never true, the robot stalls. Perception quality *is* task quality.

5 How the camera provider actually sees

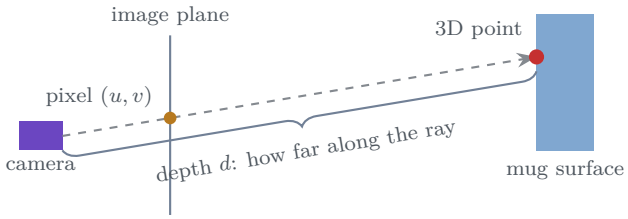
5.1 From pixels to a fact



In plain words

SAM 3 is a model that "colours in" the pixels belonging to whatever you name. It paints the mug. The depth image says how far away each pixel is. Combine the two and you know *where in the room* the mug's visible surface is. Simple geometry then answers questions like "is it higher than the table?" or "is it touching the table?".

5.2 Back-projection: one pixel to one 3D point



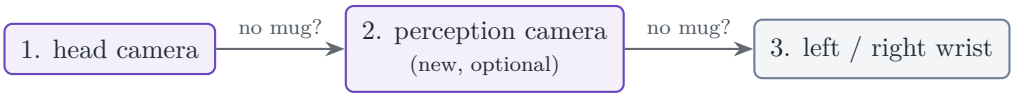
$$\mathbf{X}_{\text{cam}} = d\,K^{-1}\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \qquad \mathbf{X}_{\text{world}} = R^{\top}\left(\mathbf{X}_{\text{cam}} - \mathbf{t}\right)$$

K is the lens (intrinsic) matrix; $R, \mathbf{t}$ is the camera's pose (`world_to_camera`, OpenCV convention). In code: `(cam - t) @ R`. The provider takes the **median** over the mask's pixels so a few bad depth readings do not move the point.

Careful

A mug is hollow. Depth at the centre of the opening hits the table *behind* the mug. That is why a mask is required: the median over the mask's own pixels stays on the mug.

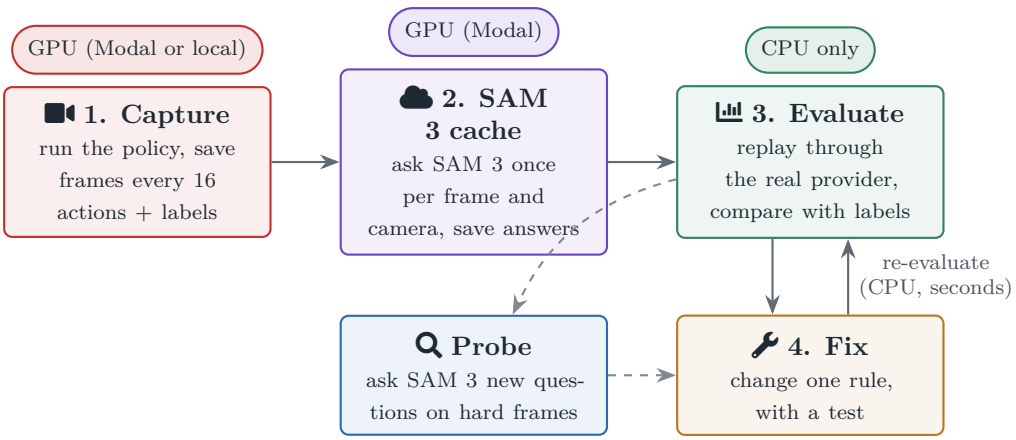
5.3 Which camera answers



The first camera with a confident, unambiguous mug wins. Wrist cameras move with the arms and see very close-up, partial views, so they are the last resort.

6 What this branch builds: an evaluation loop

Before this branch we could not say *how often* perception was right. The core of the branch is a loop that measures it, offline, on saved frames:



Key idea

The expensive steps (simulating, running SAM 3) happen **once**. After that, every idea for a fix is scored in seconds on CPU, against the same frames, with the runtime's own code. That turns "I think this is better" into "this is better on 216 frames, and here are the numbers".

6.1 Step 0: the scene viewer

`development/capture/scene_views.py` resets one seed and saves what every camera sees (RGB, depth, an overview sheet). It never moves the robot. We used it to inspect camera coverage and to aim the new perception camera.

6.2 Step 1: labelled capture — public frames and a private answer key

`observations/` (public)
`observation_00000416.npz`: RGB + depth + calibration for each camera, joint state.
Exactly what the robot sees.

`labels.jsonl` (private)
true mug pose, tilt, contacts, gripper values, and a **state**: `resting`, `held_left`, `held_right`, `staged`, `toppled`, `off_table`, ...

wall: labels never reach runtime

The labels are only read by evaluators under `development/`. The provider and planner never see them; that rule is the difference between *measuring* perception and *cheating*.

6.3 Step 2: the SAM 3 cache

SAM 3 runs on a Modal GPU. `sam3_cache.py` sends each frame's images once and stores the raw answers in `sam3_cache.json` next to the frames. Later, `CachedSam3Worker` plays them back through the *real* `ModalSam3Detector` client, so camera order and thresholds are exactly the runtime's.

6.4 Step 3: the evaluator

`perception_evaluation.py` replays every frame through the provider and compares each predicate answer with the label's truth:

truth \ provider said	true	false	unknown
true	correct	missed	cautious
false	**false true**	correct	cautious

Precision = correct trues / all trues said
Recall = correct trues / all true frames
Unknown rate = how often it says "?"
The red cell is the one we must keep at zero.

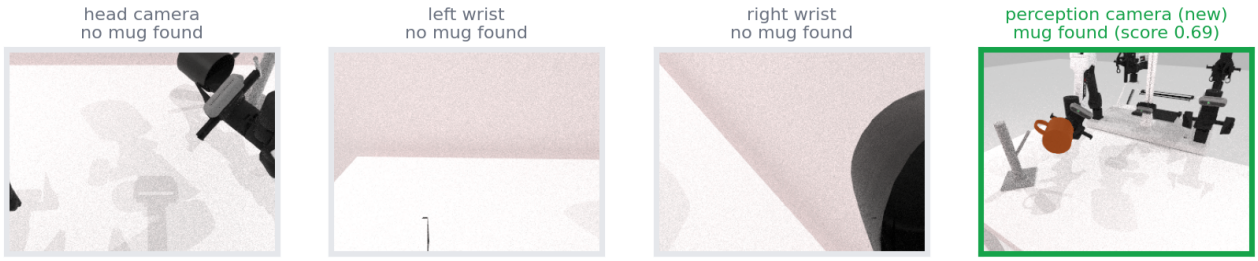
Careful

Frames, not seeds, are the unit. The policy is not deterministic: the same seed gives a different episode each time. So we compare fixes on the *same saved frames*, never by re-running a seed and hoping it repeats.

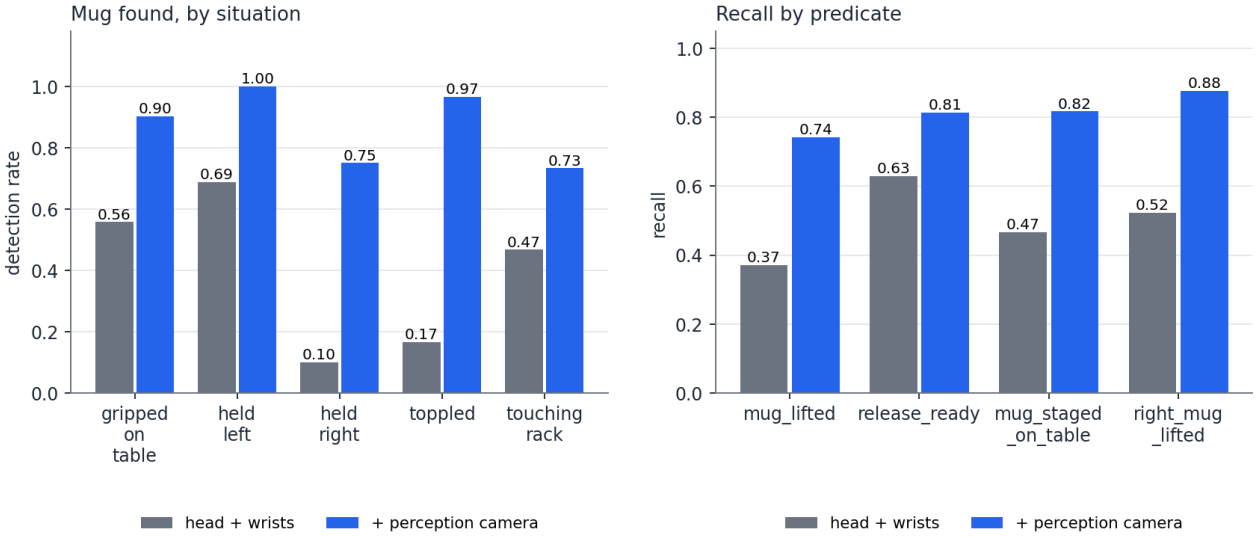
7 The fixes, one by one

7.1 Fix 1: a perception-only camera

The head camera looks down from behind the robot; when an arm lifts the mug, the arm often hides it. We added a fixed RGB-D camera, `perception_camera`, on the far side of the table (`harness/environment/perception_cameras.py`). The policy never receives it and Opus prompts don't use it; it is added after each reset so every seed keeps its exact scene.



Real frame (seed 0, action 416, mug in the right gripper): SAM 3's mug mask in orange. Only the new camera finds it.



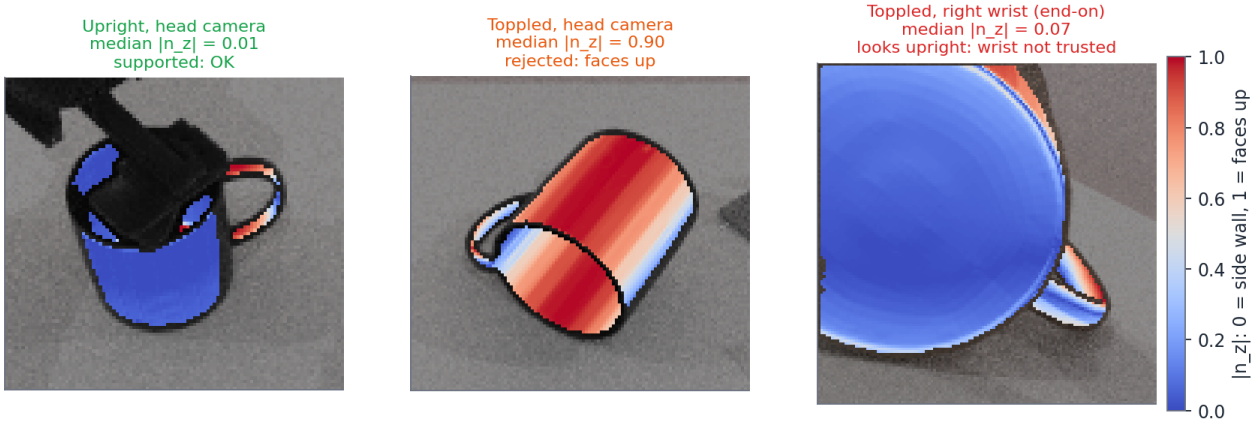
Same 216 frames scored with and without the new camera (`-ignore-camera perception_camera`). Precision stays 1.00 in both.

7.2 Fix 2: a toppled mug is not "standing on the table"

The baseline said `release_ready = true` for **39 of 56** toppled frames: a mug lying on its side *is* touching the table. Height and shape features overlapped between upright and toppled mugs. What separates them is **which way the visible surface faces**.



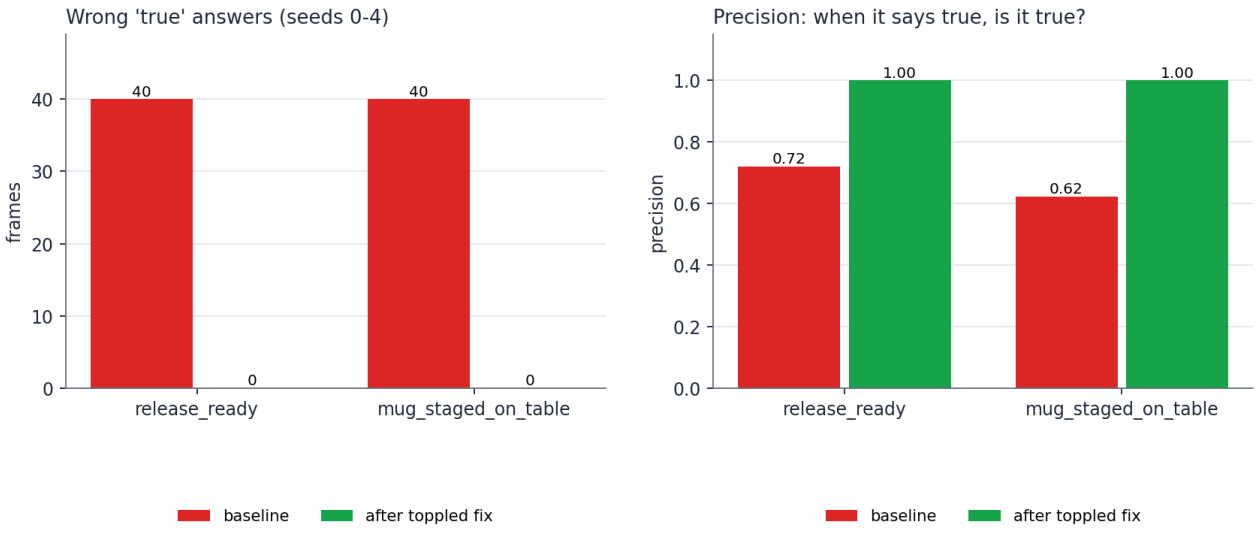
For each mask pixel we take the surface normal from neighbouring depth points (a cross product of the two directions along the surface), rotate it into the world, and keep $|n_z|$, its vertical part. The median over the mask decides: $\leq 0.25 \Rightarrow$ upright; otherwise `mug_surface_faces_up_not_upright`. Fewer than 300 usable pixels $\Rightarrow$ unknown.



Real masks coloured by $|n_z|$. The third panel is why the rule has a second part.

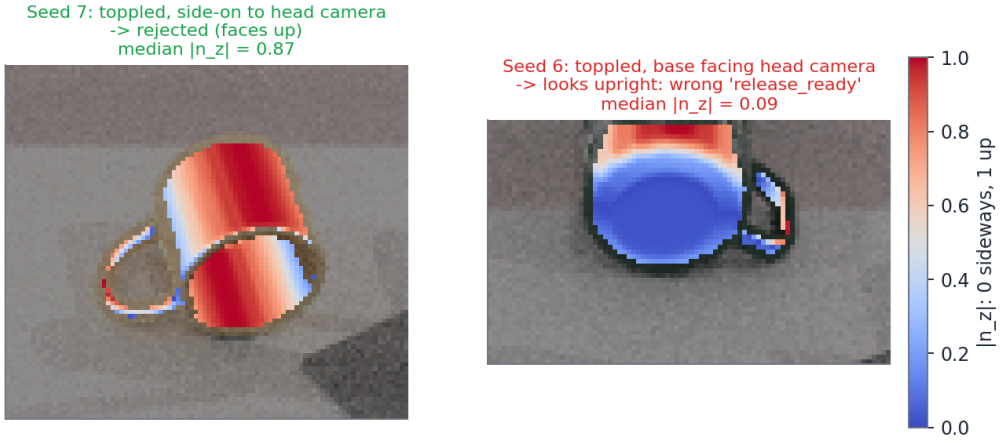
Careful

The fix did not generalise at first. On new episodes, a wrist camera looking at a toppled mug *end-on* sees mostly its flat base, which faces sideways — just like an upright wall (median 0.04–0.10). So table support is now judged **only from the head or perception camera**; a wrist-grounded mug gives `table_support_not_judged_from_wrist_view` (unknown). Every wrist-view `release_ready = true` in all reports had been wrong, so this costs no real recall.



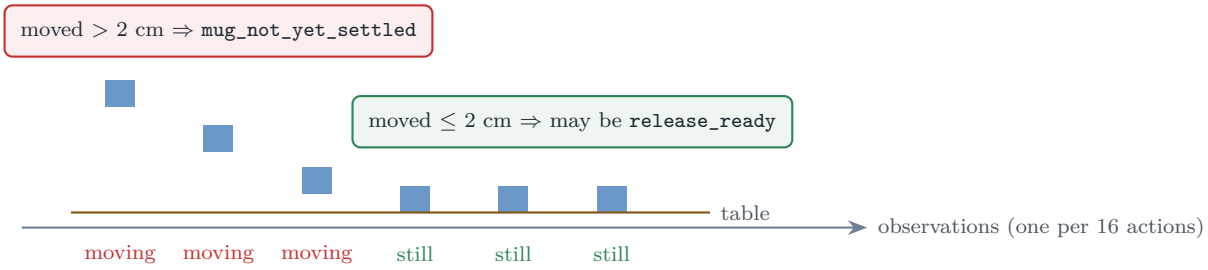
Careful

Held-out check (seeds 5–9): the head camera can see a toppled mug end-on too. In seed 6 the mug fell over with its flat base facing the head camera and lay still for the rest of the episode. Its visible surface faces sideways (median $|n_z|$ 0.09), exactly like an upright wall, so table support said "upright" in all 26 frames — 25 wrong `release_ready = true`. Seed 7's toppled mug, side-on to the same camera, was rejected every time (0.87). The normal test separates *side-on* toppled mugs, not *end-on* ones, from any camera. Every other held-out frame kept precision 1.00.



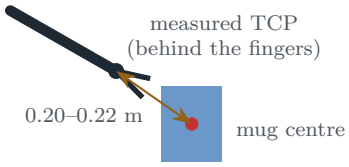
Same camera, same rule, frame 640 of two held-out episodes.

7.3 Fix 3: wait until the mug stops moving



A mug being set down can briefly "meet the table" in one frame while still moving. In the captures, mugs being lifted or lowered moved 2.6–4.0 cm between observations, while 95% of resting mugs moved less than 2 cm. So table support now needs two observations in a row that agree within 2 cm (`MUG_SETTLED_MAX_STEP_M`).

7.4 Fix 4: recognising the right-arm lift



The arm's measured pose is its wrist frame, and the policy grips the rim or handle. The old rule only linked a mug to the right arm within **0.16 m**, so it almost never fired. Now: **0.24 m** (`RIGHT_ARM_ASSOCIATION_MAX_M`), still guarded by the lift and table checks. A frame where the mug is hidden no longer resets the count, and the evaluator's truth now latches like the planner.

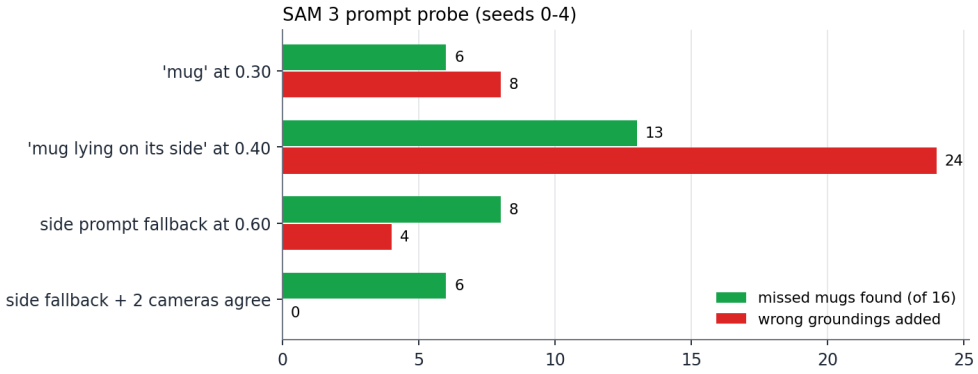
Result: `right_mug_lifted` recall 0.09 → 0.88 with the camera, precision 1.00.

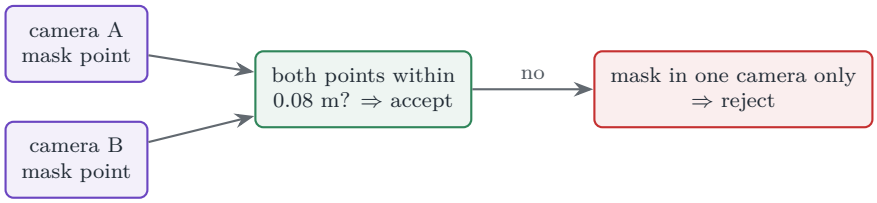
7.5 Fix 5: a mug on the floor is `off_table`, not toppled

Most remaining "toppled" misses were seed 4's mug lying on the *floor*, about 0.70 m below the table and out of every view. That is a labelling problem, not a perception one. Frames where the mug is 0.20 m or more below its start height are now labelled `off_table`. Toppled detection with the camera: **28 of 29**.

7.6 Probe: can a different SAM 3 question find sideways mugs?

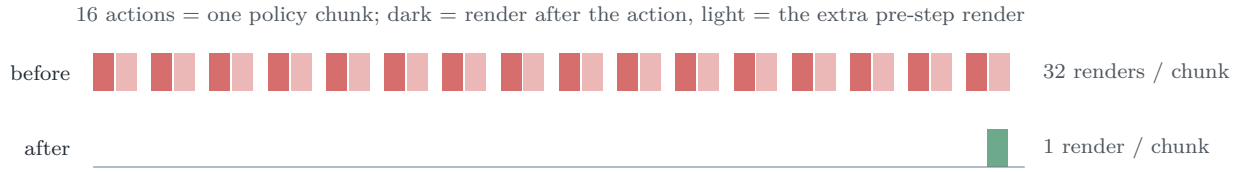
The last real misses (16 frames) are mugs turned on their side in a gripper or at the rack. The probe asks SAM 3 five prompts ("mug", "cup", "coffee mug", "mug held by a robot gripper", "mug lying on its side") with scores down to 0.05, and checks each mask's 3D point against the true mug (hit within 0.10 m).



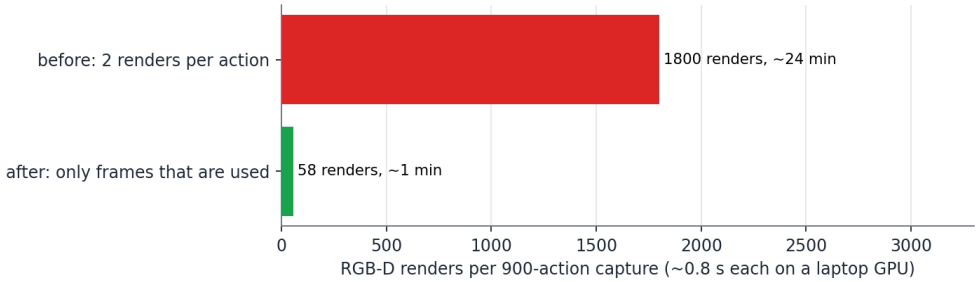


Every single-camera option added wrong groundings 0.2–0.6 m away (gripper, rack, other arm). One combined rule — fall back to "mug lying on its side" only when no camera finds "mug", and accept it only where **two cameras agree** — found 6 of 16 with no wrong answers. Because it was chosen by searching on these same frames, it is **frozen and not adopted** until it passes on held-out seeds.

7.7 Speed: render only the frames that are used



Rendering every camera with depth costs most of a simulator step (about 80% even on a Modal A10; about 0.8 s on a laptop RTX 2050). The capture now applies actions with the local client's new `advance` method (no render) and renders only the frame the policy needs and the frames it saves. Physics is unchanged: RoboTwin's `get_obs` only reads the cameras.



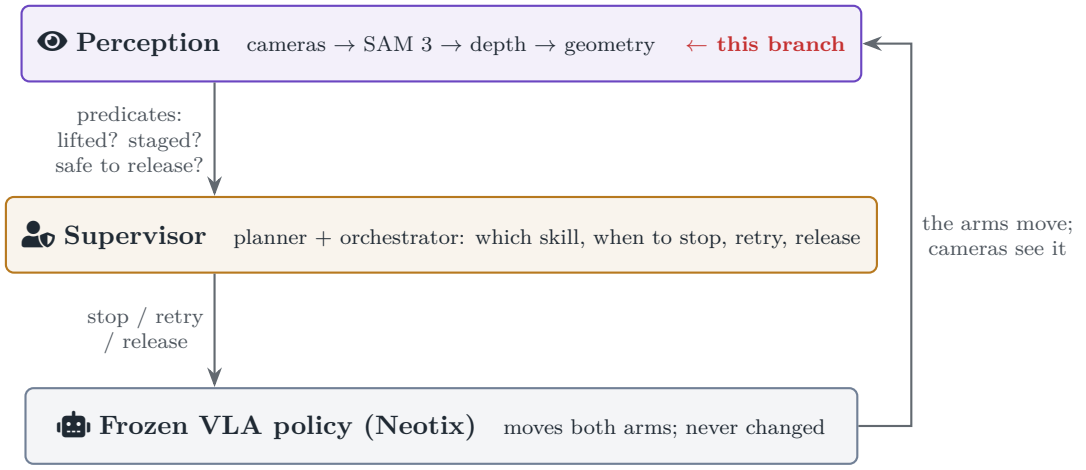
Why there were two renders per step. The base client's `step` calls `observe()` to read the previous sequence number — free for the HTTP client, which returns a cached frame. The local client's `observe()` re-renders, because analytic skills move the robot with `tick()` and a cached frame would go stale. Together: render, discard, act, render. The new run setting `render_once_per_step: true` takes those values from the client's own state instead (same observations, one render). It is off by default so existing timings stay comparable.

7.8 Smaller fixes along the way

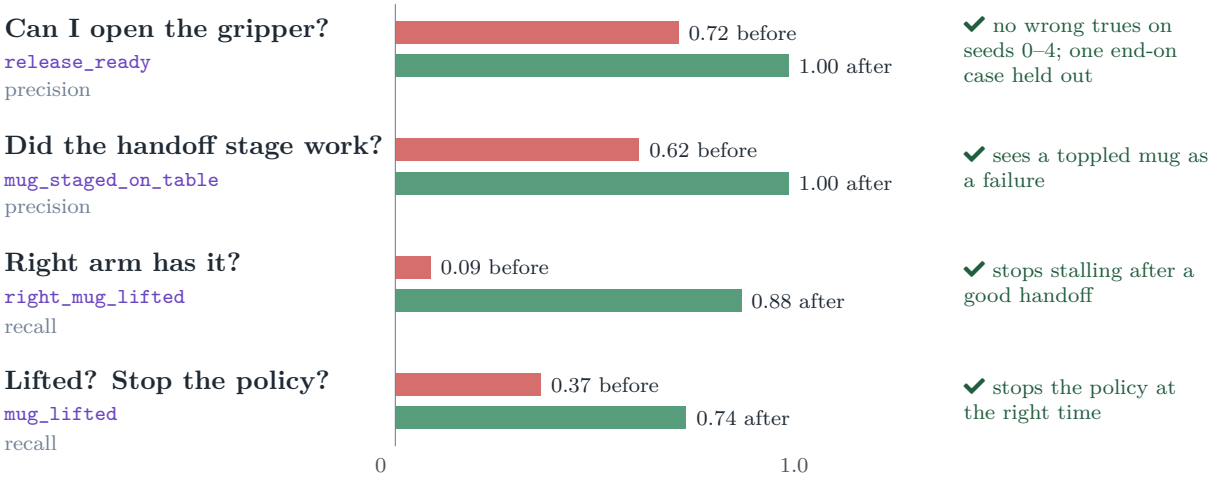
- **Invalid depth pixels** (non-finite or negative) now count as missing depth instead of rejecting the whole observation — that had crashed a capture.
- **Modal downloads** wait until the runs Volume shows every file.
- **One failed seed** no longer stops a capture batch.
- **GPU fallbacks**: `SIM_GPU=L4,A10G,T4` tries GPUs in order; the manifest records which one ran.
- **Labels**: a mug held tilted in a gripper is not `toppled`.
- **Policy-call timeout.** During the held-out capture the policy container vanished mid-request and Modal's `.remote()` waited two hours on a closed connection. Every policy call now gives up after 5 minutes (`POLICY_CALL_TIMEOUT_SECONDS`); the seed fails cleanly and the loop moves on. It does not retry: a fresh container has lost the policy's last two observations.

8 Why this matters: perception is the supervisor's eyes

The project's supervisor is the orchestrator plus a planner (scripted or Opus), sitting on top of the frozen Neotix policy. It decides when to let the policy act, when to stop it, when to retry, and when it is safe to let go of the mug. Every one of those decisions is made from predicates — and in a real run, predicates come only from perception. If the eyes are wrong, the supervisor is confidently wrong.

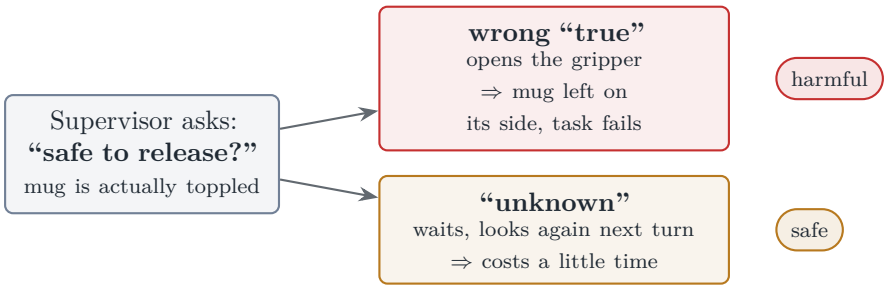


8.1 What each fix gives the supervisor



Seeds 0–4. The first two compare the baseline with the toppled fix; the last two compare without and with the perception camera, all fixes on.

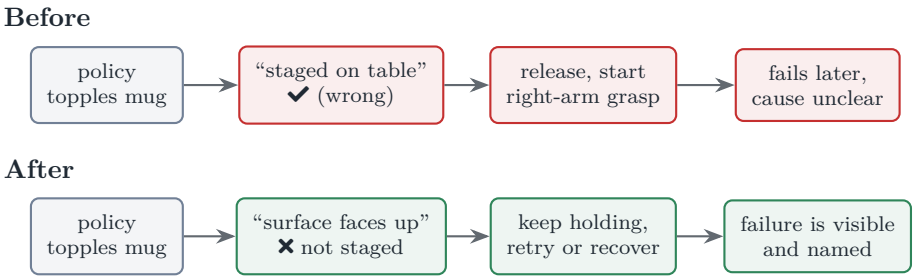
8.2 Idea 1: a supervisor must intervene safely



A supervisor is only worth having if it does not make the policy worse. That is why every fix aims to keep `release_ready`, `mug_staged_on_table` and `right_mug_lifted` at **precision 1.00**: the supervisor should never act on a false fact; lower recall only makes it more patient. On the development seeds all three reached 1.00. On the held-out seeds `right_mug_lifted` stayed at 1.00, but a toppled mug seen end-on by the head camera gave 25 wrong `release_ready` answers (Section 7.2) — exactly the kind of error the held-out test exists to catch.

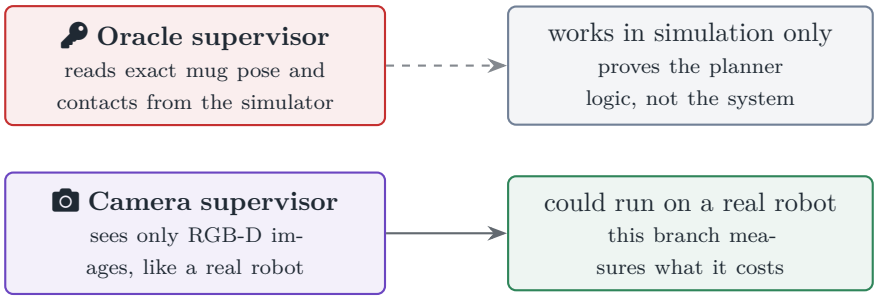
8.3 Idea 2: a supervisor must notice failure

The policy topples, drops or misses the mug and does not notice. Before this branch, the supervisor did not notice either.



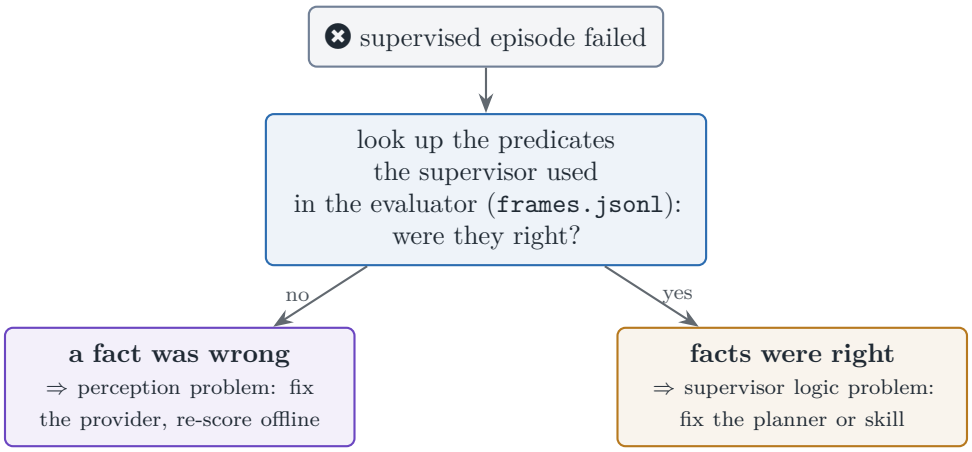
The settle rule does the same for a mug that is still moving: “touching the table” in one frame is no longer enough to call it placed.

8.4 Idea 3: no cheating — the supervisor must work without the answer key



Until now the supervisor could be shown working only with the oracle. This branch shows how well it can run on camera evidence alone, and puts a number on the gap. That is what makes “an agent-aware supervisor improves a frozen VLA” a credible claim.

8.5 Telling a perception error from a supervisor error



8.6 What the supervisor still cannot see well

Sideways mugs at the rack
often "unknown": the supervisor waits or stops instead of guessing — safe, but can cost success

Rack predicates
`mug_near_rack`, `rack_seat_aligned` are not yet scored by this evaluator — next for the HANG phase

Cost per check
every check is a SAM 3 call: perception runs at turn boundaries, and render speed matters for live runs

Key idea

Perception does not make the supervisor smarter; it makes the supervisor's facts *trustworthy enough that its interventions help instead of hurt.* The held-out seeds 5–9 test whether that still holds on episodes no rule was tuned on.

9 The commands, in order, and why



Careful

One GPU job at a time. The deployed policy keeps a single observation history, so two captures or episodes at once would mix their histories.

Preview a camera (local, no policy)

```
python -m development.capture.scene_views --seed 3 \
--output data/scenes/seed_3_cam --perception-camera default
```

Why: see what each camera covers before spending GPU time.

Capture labelled frames (seeds)

On Modal (fast, uses Modal units for the simulator GPU and the policy):

```
SIM_GPU=L4,A10G,T4 modal run modal_apps/simulator.py::frames \
--seeds 0,1,2,3,4 --perception-camera default \
--output-root data/perception/frames_cam
```

Locally (now a few minutes per seed; only the policy uses Modal):

```
for s in 6 7 8 9; do
conda run -n robotwin python -m development.capture.labeled_frames --seed $s \
--output data/perception/frames_holdout/seed_$s --perception-camera default
done
```

Why: a seed fixes the starting scene; the policy then runs up to 900 actions, and a frame plus a label is saved every 16 actions (≈58 frames, ≈45 MB per seed).

Deploy SAM 3 and cache its answers

```
modal deploy modal_apps/sam3.py
python -m development.replays.sam3_cache --frames data/perception/frames_cam/seed_0
```

Why: pay for SAM 3 once per frame and camera; every later evaluation is free.

Evaluate (CPU, seconds)

```
python -m development.replays.perception_evaluation \
--frames data/perception/frames_cam/seed_0 ...  --frames .../seed_4 \
--output data/perception/reports/with_camera
# same frames as if the new camera did not exist:
...  --output data/perception/reports/without_camera --ignore-camera perception_camera
```

Why: writes `summary.md` (detection by state, precision/recall per predicate) and `frames.jsonl` (every frame's answer and reason).

Probe SAM 3 prompts

```
python -m development.replays.sam3_mug_probe --query --frames .../seed_0 ...  # GPU
python -m development.replays.sam3_mug_probe --frames .../seed_0 ...  \
--runtime-report data/perception/reports/with_camera/frames.jsonl  # CPU
```

Why: test new prompts or thresholds on exactly the frames the runtime misses, without changing the runtime.

Held-out seeds (5–9): capture, cache, evaluate, probe

```
for s in 5 6 7 8 9; do
conda run -n robotwin python -m development.capture.labeled_frames --seed $s \
--output data/perception/frames_holdout/seed_$s --perception-camera default || echo
"seed $s failed"
done
# then the same loop with:  python -m development.replays.sam3_cache --frames
.../seed_$s
python -m development.replays.perception_evaluation --frames .../seed_5 ...  --frames
.../seed_9 \
--output data/perception/reports/holdout_with_camera
```

Why: every threshold was chosen on seeds 0–4; seeds 5–9 are scored with the rules frozen. If a seed fails or hangs, move its folder aside (`mv .../seed_6 .../partial_seed_6`) before rerunning it: the capture never writes into an existing folder. The cache step skips seeds that already have a cache.

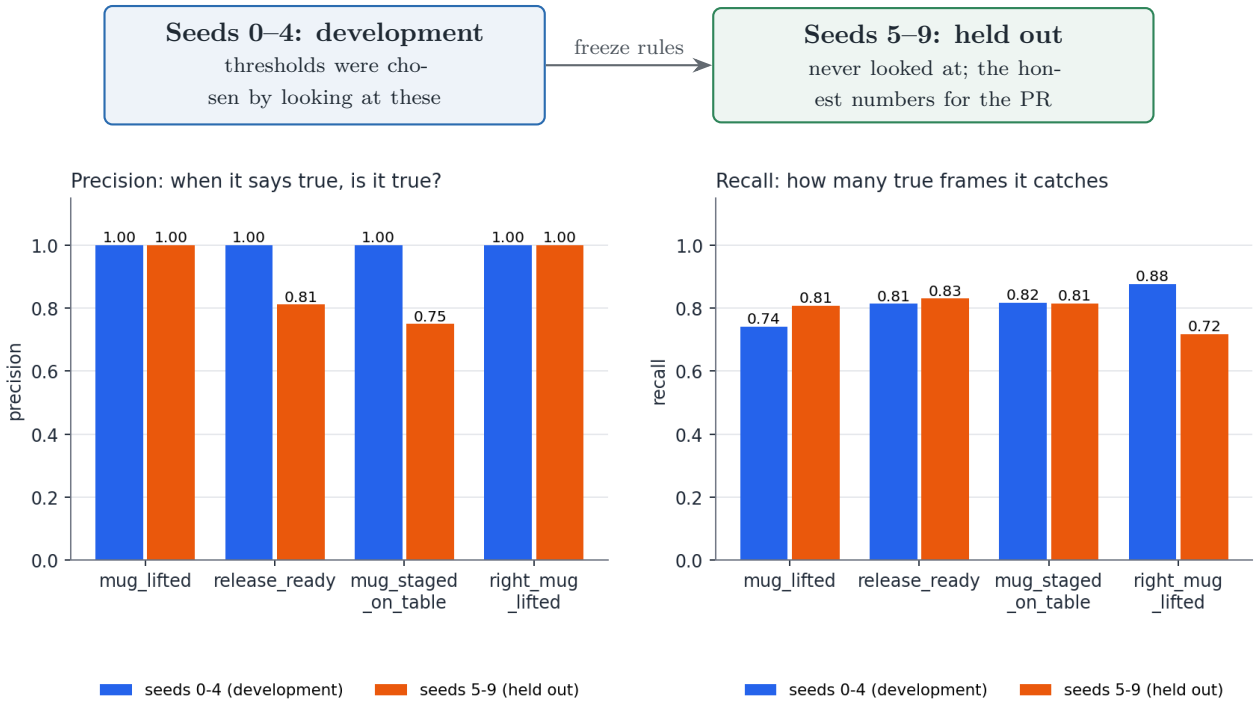
Tests

```
conda run -n robotwin python -m pytest -q tests
```

10 Where we are, and what is next

Predicate	Baseline seeds 0–4	Fixes, no camera seeds 0–4	Fixes + camera seeds 0–4	Held out + camera seeds 5–9
`mug_lifted`	1.00 / 0.71	1.00 / 0.37	1.00 / 0.74	1.00 / 0.81
`release_ready`	0.72 / 0.79	1.00 / 0.63	1.00 / 0.81	0.81 / 0.83
`mug_staged_on_table`	0.62 / 0.70	1.00 / 0.47	1.00 / 0.82	0.75 / 0.81
`right_mug_lifted`	1.00 / 0.09	1.00 / 0.52	1.00 / 0.88	1.00 / 0.72

Each cell is precision / recall. Baseline: 236 frames, first capture, head + wrists. Middle two: 216 frames of a second capture, scored without and with the new camera. Held out: 267 frames of seeds 5–9, rules frozen. Different episodes in each capture, so compare columns of the same capture.



With the perception camera. All 25 held-out wrong trues come from the one end-on toppled mug in seed 6; without that episode both predicates keep precision 1.00 (recall 0.83 and 0.81).

What the held-out seeds confirmed: `mug_lifted` and `right_mug_lifted` keep precision 1.00 (the 0.24 m association held; right-arm recall fell to 0.72, still far above the 0.09 baseline); the settle rule caused no wrong trues; the perception camera again lifts every recall (held-right detection 0.32 → 0.68). **What it broke:** the toppled test, for a mug lying with its base towards the head camera.

✔ Captured seeds 5–9 with the camera, cached SAM 3, evaluated with every rule frozen.

1. Fix the end-on toppled case *using development seeds only*, then confirm it on fresh seeds (10–14): seeds 5–9 have now been looked at, so tuning on them would no longer be an honest test.
2. Probe seeds 5–9 (running): adopt the frozen two-camera rule only if it still finds misses with zero wrong groundings.
3. Decide whether the perception camera and `render_once_per_step` are on by default, using measured render cost.
4. One PR from `perception/grounding-evaluation` with the before/after tables.

Glossary

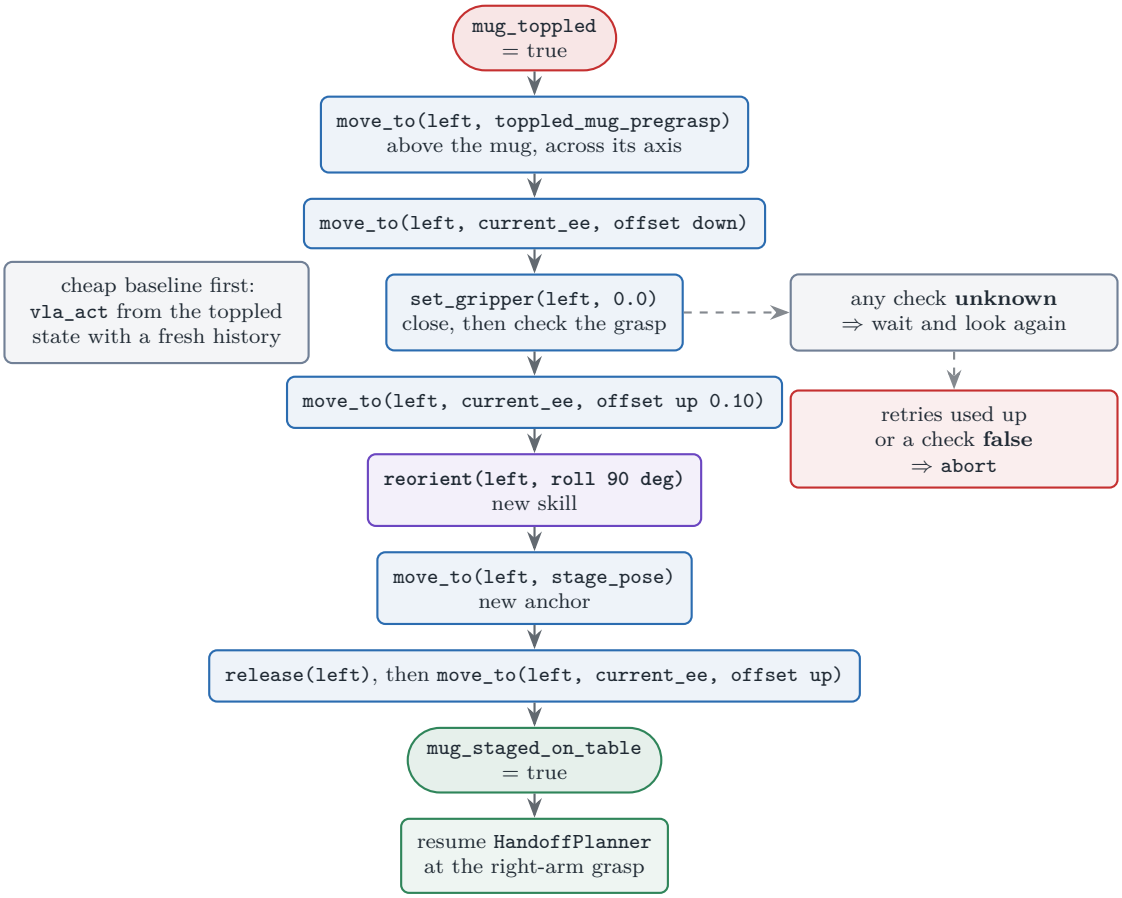
Predicate	a named true/false/unknown fact the planner and skills rely on
Provider	the object that answers predicates (camera or oracle)
Planner	chooses one skill per turn from images and predicates
Skill	a bounded robot action (`vla_act`, `move_to`, `set_gripper`, ...)
Oracle	answers from exact simulator state; development only
SAM 3	segmentation model: paints the pixels of a named object
Mask	the set of pixels SAM 3 assigned to an object
Grounding	turning a mask plus depth into a 3D position
Latch	once true, stays true for the episode
Precision / recall	is "true" right? / how many true frames were caught?
Seed	fixes the starting scene; the policy still varies
Held-out	frames never used to choose a threshold
TCP	tool centre point: the arm's measured end-effector frame

A Plan: recovering a toppled mug

Key idea

Goal: when the policy topples the mug, the supervisor sends bounded skill commands that pick the mug up, stand it upright at the place where the left hand normally sets it down, and then continue the normal handoff. **This is a plan; none of it is implemented yet.**

A.1 The recovery as a chain of skills and checks



A.2 What exists and what is missing

✔ Already in the code

- skills: `move_to` (named target + offset $\leq$ 20 cm), `rotate_wrist` (yaw $\pm 30°$), `set_gripper`, `release`, `vla_act`, `abort`
- anchors: `current_ee`, `mug_pregrasp`, `rack_approach`, `table_approach`, `rack_seat`
- `HandoffPlanner` phases and predicate gates
- CuRobo planning to any full pose (`plan`)
- the surface-normal toppled evidence (as a reason)
- checkpoint bank and `manual_skill.py` for rehearsing skills

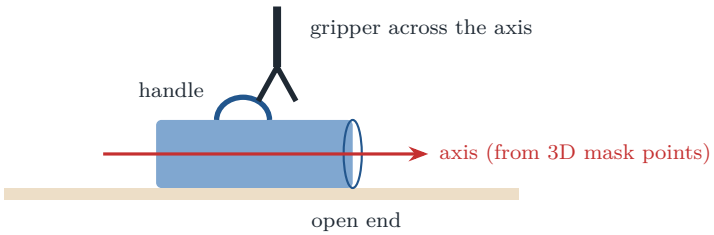
✚ To build

1. `mug_toppled` predicate (needs the end-on fix first)
2. `toppled_mug_pregrasp` anchor: axis direction and grasp from depth
3. `stage_pose` anchor: the handover place, defined publicly
4. `reorient` skill: about 90° roll through CuRobo
5. planner recovery phase (+ Opus schema, new prompt version)
6. toppled checkpoints and a recovery evaluation

A.3 The perception pieces

1. A `mug_toppled` predicate. Today "toppled" only blocks `release_ready`. Recovery needs a fact the planner can branch on: true when the mug is grounded from the head or perception camera, touches the table and its surface faces up. It inherits the end-on blind spot from seed 6 (it would say false there), so that fix comes first — designed on seeds 0–4, confirmed on fresh seeds 10–14.

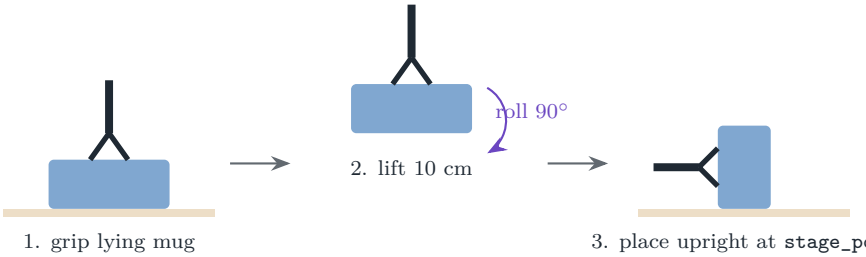
2. Where it lies and which way. A pick-up needs the mug's axis and its open end, not just its centre.



The long direction of the mask's 3D points gives the axis (it lies horizontal); the end whose depth shows a hollow gives the opening. Trusted perception code turns that into a fresh `toppled_mug_pregrasp` anchor each observation — never simulator poses.

3. A public "handover place". The left hand sets the mug where the *policy* chooses, near RoboTwin's `middle_pos` — simulator truth, so not usable at runtime. Define `stage_pose` as a fixed pose in the table frame, taught once and stored in config (like a taught position on a real robot), or as the median position where perception saw the mug when `mug_staged_on_table` first became true in successful development runs, frozen into config.

A.4 The motion piece: standing it up

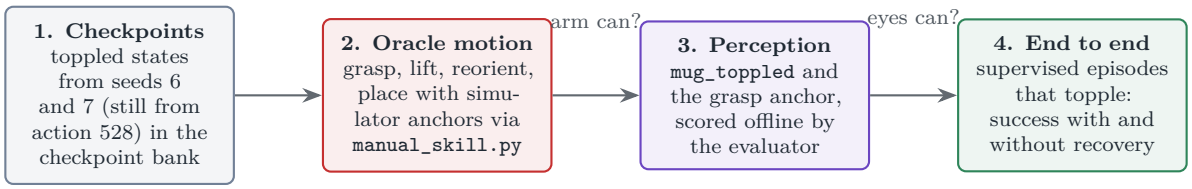


`rotate_wrist` only turns ±30° about the vertical. Standing a mug up needs about 90° about a horizontal axis: a new bounded `reorient` skill (axis and angle, angle limited to about ±90°, planned with CuRobo to a full pose), or `move_to` accepting a named target orientation such as "upright at `stage_pose`".

Careful

Check feasibility first. The Aloha arms have 6 joints, so a 90° roll above the table may not be reachable everywhere. The gripper opening against the mug's ≈8 cm diameter decides whether the body can be gripped or the rim at the open end must be pinched (as a person would). If the arm cannot do this with perfect positions, the perception work for it is not worth building yet.

A.5 How to develop it safely



Step 2 separates "can the arm do it?" from step 3's "can perception see it?". Each new predicate is scored for precision before the planner may act on it, as for every predicate in this branch.

A.6 Effort and risk

Piece	Effort	Risk
`mug_toppled` predicate	small, after the end-on fix	medium (end-on views)
Axis and grasp anchor from depth	medium	medium
`stage_pose` anchor from config	small	low
`reorient` skill with CuRobo	medium–large	**high** (reachability, gripper fit)
Planner phase and Opus schema	medium	low

Start with steps 1 and 2 — checkpoints and oracle motion — before any perception code.